

Practice : DP1--Weighted Interval Scheduling

Deadline: May.20/21/22. Must be checked onsite (during labs).Online checks via email or messaging apps are not accepted.

1. Objective

This Practice has one question based on **Chapter 6.1 - 6.2 Weighted Interval Scheduling** in the lecture. It is the basic exercise of Dynamic Programming.

2. Task

Description:

Job j starts at s_j , finishes at f_j , and has weight or value v_j .

Two jobs compatible if they don't overlap (in time).

Goal: find maximum weight subset of mutually compatible jobs.

Input Description:

The first line contains an integer n , representing there are n jobs in total.

Then the following n lines represent each job, which contains three integer s , e , w and separated by a space.

- s means the start time.
- e means the end time.
- w means the weight.

Output Description:

The first line has an integer, representing the maximum weight of jobs.

Then the following lines contains the selected jobs in descending order of start time with the format as: `(start_time-end_time) weight`

Input Test case1:

```
7
0 7 2
5 9 6
8 10 4
1 4 5
3 6 3
9 19 6
7 11 10
```

Output1

```
17
(9-19) 6
(5-9) 6
(1-4) 5
```

Input Test case2:

```
50
13 34 3
24 34 19
34 57 5
11 30 15
96 109 5
107 113 1
9 31 6
83 91 9
45 74 4
94 97 11
98 109 5
3 18 19
5 21 14
45 65 11
64 69 9
113 134 2
104 109 3
17 28 9
62 83 5
17 40 17
54 84 10
66 70 20
40 63 13
5 34 16
7 10 4
107 115 8
64 76 4
72 76 5
95 124 15
```

```
52 72 2
88 116 4
56 72 9
18 45 14
33 43 17
41 70 6
113 115 4
55 65 12
24 54 14
21 37 3
86 113 11
68 91 11
81 107 16
22 52 3
1 23 15
23 30 4
39 66 19
42 44 8
67 95 7
92 116 5
35 42 7
```

Output2

```
119
(113-115) 4
(98-109) 5
(94-97) 11
(83-91) 9
(72-76) 5
(66-70) 20
(55-65) 12
(42-44) 8
(35-42) 7
(24-34) 19
(3-18) 19
```

3. Introduction

3.1 Solution of this question

In this part, using test case 1 as an example to introduce the solution.

- Sort the jobs in ascending order by their **end times**.

Index	0	1	2	3	4	5	6
End Time	4	6	7	9	10	11	19
Start Time	1	3	0	5	8	7	9
Weight	5	3	2	6	4	10	6

- For each job, find the **last job** that does not overlap with it. Record this in an array $p[i]$.

Index	0	1	2	3	4	5	6
End Time	4	6	7	9	10	11	19
Start Time	1	3	0	5	8	7	9
Weight	5	3	2	6	4	10	6
$p[i]$	-1	-1	-1	0	2	2	3

- After sorting, for the current job j , find the two values:
 - If the total optimal weight doesn't contain the current job, $OPT(j) = OPT(j - 1)$.
 - If the total optimal weight contains the current job, $OPT(j) = OPT(p[j]) + w_j$
- Compare the two values of the step above, and record the max value of $OPT(j)$. The $OPT(n - 1)$ is the answer.

Index	0	1	2	3	4	5	6
End Time	4	6	7	9	10	11	19
Start Time	1	3	0	5	8	7	9
Weight	5	3	2	6	4	10	6
$p[i]$	-1	-1	-1	0	2	2	3
$OPT(j)$	5	5	5	11	11	15	17

3.2 Improved by binary Search

When building the array $p[i]$, how to find the **last job** that does not overlap with the current job?

In this case, we recommend using **binary search** to improve the time.

4. Submission

Show the following to your tutor or SA **onsite**:

Grading:

- Submissions on time:

- 0.5 points for finding correct weight of two test cases.
 - 0.3 points for finding final selected jobs of two test cases
 - 0.2 point for using binary search to find $p[i]$
- Late submissions: 0 points.